

Asignatura Full Stack I: Directrices para Escoger Tema del Proyecto Semestral

Introducción

Para asegurar un estándar académico equivalente en todas las sedes, este proyecto de tercer semestre **debe cumplir los mínimos obligatorios indicados en este documento**, permitiendo aumentar la complejidad opcionalmente. El docente guiará a los estudiantes para establecer límites realistas que aseguren componentes funcionales en los plazos establecidos. Las tecnologías y métodos **son referenciales y ajustables por el docente**, manteniendo siempre los requisitos mínimos.

Contexto del proyecto

El proyecto debe **resolver un problema real** mediante **un sistema de gestión completo o servicio digital** que supere ejercicios académicos simples, aplicando el dominio de programación y bases de datos propio de tercer semestre. Para ello, debe **integrar múltiples módulos funcionales y componentes backend** que justifiquen técnicamente el uso de una arquitectura de microservicios, evitando soluciones donde una arquitectura monolítica sería suficiente.

Arquitectura del sistema

El proyecto exige el uso obligatorio de una arquitectura de microservicios con **Spring Boot**, garantizando un diseño completamente desacoplado donde cada componente sea independiente y autosuficiente. Se debe implementar un mínimo de **10 microservicios**, cada uno con una responsabilidad única, lógica de negocio propia y una interfaz de consumo estandarizada. Los requisitos técnicos obligatorios son:

- **Independencia:** Cada microservicio debe estar desacoplado, manejar su propia lógica y tener una responsabilidad clara.
- **Consumo de APIs:** Exposición de APIs REST con endpoints funcionales siguiendo la jerarquía de entidades.
- **Estructura de URLs:** Las rutas deben seguir principios RESTful, manteniendo consistencia en el uso de recursos, jerarquía de entidades y parámetros (Protocolo + Dominio + Puerto + Ruta + Parámetros).
- **Escalabilidad:** El número de 10 microservicios es la base mínima, permitiendo añadir más según las necesidades del sistema.

Comunicación entre microservicios

El sistema debe garantizar una **interacción real y fluida entre sus componentes**, permitiendo la consulta, validación y transferencia de datos a través de los distintos módulos por medio de sus microservicios. Para lograrlo, es fundamental implementar mecanismos de comunicación robustos y herramientas de orquestación que aseguren la cohesión y disponibilidad del ecosistema de microservicios. La interacción y orquestación del sistema debe incluir:

- **Comunicación híbrida:** uso de OpenFeign para llamadas sincrónicas y Apache Kafka para comunicación asincrónica basada en eventos, orientada al desacoplamiento y consistencia eventual.
- **Flujo de Información:** Implementación de consultas y validaciones cruzadas que aseguren la consistencia de los datos entre servicios.
- **API Gateway:** Establecimiento de un punto de entrada único que centralice y gestione todas las solicitudes externas.
- **Service Discovery (Eureka):** Uso de un directorio dinámico para la localización automática y gestión del estado de los microservicios activos.

Persistencia de datos

El sistema exige independencia total a nivel de datos, obligando a que **cada microservicio gestione su propia base de datos persistente e independiente**. Este diseño busca garantizar la autonomía del servicio, la integridad de la información y la aplicación estricta de reglas de negocio desde la capa de aplicación, evitando dependencias estructurales compartidas. Para la gestión de persistencia e integridad considerar:

- **Autonomía de Datos:** Prohibido compartir tablas; se deben usar proyecciones sincronizadas (Kafka) para replicación de datos y proyecciones de JPA para optimizar el acceso a la información y reducir la carga de consultas.
- **Modelado Relacional:** Las bases de datos deben estar **normalizadas**, con entidades y relaciones validadas mediante **diagramas de modelo relacional**.
- **Motores Soportados:** Uso obligatorio de **MySQL** (recomendado), **Oracle** o **PostgreSQL**.
- **Entornos de Gestión:** Implementación local mediante motores de base de datos instalados o contenedores (Docker), pudiendo utilizar herramientas como XAMPP, Laragon o equivalentes.
- **Calidad de Datos:** El proyecto debe evidenciar integridad referencial, entidades bien modeladas y validaciones de negocio rigurosas.

Roles de usuario

El sistema requiere la implementación de un control de acceso basado en **2 a 3 roles de usuario claramente diferenciados**. Esta segmentación asegura que cada tipo de usuario posea una experiencia personalizada y restringida, interactuando exclusivamente con las funciones y módulos que le corresponden según sus responsabilidades dentro del servicio digital. Cada rol y permiso debe cumplir:

- **Funcionalidad Específica:** Cada rol debe tener acciones exclusivas y tareas asignadas dentro de la lógica del sistema.
- **Gestión de Privilegios:** Implementación de distintos niveles de acceso para proteger la integridad y seguridad de la información.
- **Interacción Modular:** Los roles deben navegar e interactuar con diferentes módulos backend según su perfil.
- **Perfiles de Ejemplo:** Uso de categorías como **Administrador** (gestión total), **Usuario Cliente** (consumo) u **Operador** (gestión operativa).

Funcionalidades del sistema

El sistema debe integrar una lógica de negocio robusta que justifique el uso de microservicios, asegurando que **las APIs permitan una gestión integral de todos los procesos digitales**. Cada componente debe ir más allá de un almacenamiento pasivo, ejecutando procesos especializados y validaciones estrictas que garanticen la calidad y consistencia de la información circulante. Las capacidades funcionales del sistema deben incluir:

- **Operaciones CRUD y lógica extendida:** Implementación de mantenimientos básicos junto con procesos de negocio especializados.
- **Validación de Datos:** Uso obligatorio de **DTOs** y **Spring Boot Starter Validation** para asegurar la integridad antes de procesar.
- **Consultas Optimizadas:** Empleo de **Query Methods** y **Custom Queries** en los repositorios para búsquedas eficientes y personalizadas.
- **Lógica y Flujo:** Aplicación estricta de reglas de negocio y sincronización de información entre los distintos microservicios.

Seguridad básica del sistema

El sistema debe integrar una capa de seguridad sólida que garantice la protección de los datos y el acceso controlado a los recursos. Utilizando el ecosistema de **Spring Security**, se busca implementar un flujo de autenticación y autorización profesional que resguarde la identidad de los usuarios y la integridad de la información en toda la arquitectura. Los mecanismos de seguridad requeridos son:

- **Cifrado de Credenciales:** Uso de **BCrypt** para encriptar contraseñas antes de almacenarlas en la base de datos.
- **Autenticación y Sesión:** Implementación de login con generación de **Tokens (JWT)** para manejar sesiones de forma segura y sin estado.
- **Control de Acceso Basado en Roles (RBAC):** Validación estricta de permisos mediante roles de usuario para restringir el acceso a endpoints específicos.
- **Filtros de Seguridad:** Configuración de filtros personalizados para interceptar, validar y autorizar cada solicitud al sistema.

Control de versiones y repositorio

El uso de **GitHub** es obligatorio para **centralizar el código y gestionar el ciclo de vida del desarrollo**. El repositorio debe reflejar trazabilidad del desarrollo, evidenciando la evolución del sistema que demuestre el trabajo colaborativo, la evolución constante del sistema y el aporte individual de cada integrante, asegurando la transparencia y organización del proyecto académico. La gestión de repositorio y versiones debe:

- **Evidencia de Avance:** Registro de un historial de cambios progresivo que refleje el crecimiento real del sistema.
- **Colaboración del Equipo:** Participación activa y equitativa de todos los alumnos, reflejada en las métricas de contribución.
- **Prácticas Recomendadas:** Uso de **ramas (branches)** para nuevas funcionalidades y **commits frecuentes** con mensajes descriptivos.
- **Organización:** Estructura de carpetas clara y profesional que facilite la navegación por los diferentes microservicios.

Documentación, pruebas y despliegue (Experiencia de Aprendizaje 3)

Esta fase final del proyecto asegura que el sistema sea profesional, confiable y accesible externamente. Se enfoca en validar la calidad del código mediante pruebas, documentar su funcionamiento para terceros y garantizar que los microservicios estén operativos y disponibles **a través de una URL funcional**. La documentación, calidad y lanzamiento debe cumplir con:

- **Documentación Técnica:** Uso de **SpringDoc/Swagger** para generar manuales interactivos y archivos **Markdown** para la guía del sistema.
- **Pruebas Unitarias:** Implementación de **JUnit 5 y Mockito** para asegurar que cada componente backend funcione de forma aislada.
- **Despliegue Local y Exposición:** Ejecución en entorno local (o Docker) o uso de herramientas de exposición como Ngrok o despliegue en plataformas cloud.
- **Servicios Cloud:** Opción de desplegar en plataformas gratuitas como **Railway o Render** para mantener el sistema disponible en la web.
- **Acceso API:** El resultado final debe ser una URL funcional que permita consumir los endpoints de los microservicios desde cualquier ubicación.

Ejemplos de Proyectos

Los ejemplos de proyectos definen el estándar de complejidad y arquitectura requeridos para validar las competencias de la asignatura. Se busca que los estudiantes enfrenten desafíos técnicos reales, evitando soluciones triviales que no justifiquen el uso de una infraestructura distribuida o que carezcan de procesos de negocio significativos. Respecto a la clasificación de proyectos, se debe cumplir:

- **Proyectos Aceptables:** Sistemas integrales como gestión de **reservas (hoteles/servicios)**, plataformas de **pedidos/restaurantes**, bibliotecas, **ventas online (E-commerce)**, inventarios logísticos o marketplaces.
- **Proyectos Rechazados:** Aplicaciones excesivamente simples como **listas de tareas (To-Do)**, agendas básicas u operaciones CRUD simples sin lógica de negocio significativa.
- **Exclusiones Técnicas:** No se admiten sistemas con **base de datos compartida** o aquellos que no demuestren una **interacción real** entre sus microservicios.
- **Nivel Esperado:** Propuestas que requieran múltiples módulos funcionales y flujos de información coordinados.

Herramientas y Metodologías Sugeridas

Herramienta / Técnica	¿Para qué sirve? (Explicación Sencilla)
I. Arquitectura	
Spring Boot	El framework principal que se usará para crear cada uno de los microservicios de forma rápida y estandarizada.
Microservicios Desacoplados	Técnica de diseño donde cada módulo (ej: Ventas, Inventario) es independiente, maneja su propia base de datos y no depende directamente de los otros para funcionar.
II. Base de Datos	
Base de Datos por Servicio	Patrón obligatorio donde cada microservicio tiene su propia base de datos (MySQL, PostgreSQL u Oracle), prohibiéndose compartir tablas directamente.
Spring Data JPA	Biblioteca que facilita la interacción entre el código Java y la base de datos sin escribir SQL complejo manualmente.
Diagrama Entidad-Relación (DER)	Técnica de modelado para planificar y validar cómo se estructuran los datos antes de crear las tablas reales.
III. Documentación	
SpringDoc OpenAPI / Swagger	Genera automáticamente una página web interactiva para visualizar y probar las APIs (endpoints) directamente desde el navegador.
Anotaciones de Swagger	Etiquetas especiales en el código (ej: @Operation) que añaden descripciones amigables a la documentación automática.
Markdown (README.md)	Archivo de texto guía en el repositorio que explica de qué trata el proyecto, cómo instalarlo y cómo ejecutarlo.
IV. Pruebas (Calidad)	
JUnit 5	El framework estándar para escribir y ejecutar pruebas automáticas que verifican si la lógica del código funciona correctamente.
Mockito	Biblioteca para simular comportamientos de objetos complejos (como bases de datos) y probar tus servicios de forma aislada y segura.
Spring Boot Validation	Uso de anotaciones (ej: @NotNull) en tus datos (DTOs) para asegurar que la información que recibes sea correcta antes de procesarla.

V. Comunicación	
Feign Client	Biblioteca para que los microservicios se comuniquen entre sí de forma sincrónica (esperando respuesta inmediata) mediante llamadas HTTP sencillas.
Apache Kafka	Plataforma de mensajería para comunicación asincrónica (sin esperar respuesta), usada para sincronizar datos mediante eventos entre microservicios.
Eureka (Service Discovery)	Un directorio dinámico donde cada microservicio se registra al iniciar, permitiendo que otros lo localicen automáticamente por su nombre.
API Gateway	El punto de entrada único para todas las solicitudes externas, que redirige el tráfico al microservicio correspondiente.
VI. Seguridad	
Spring Security	Framework completo para gestionar la autenticación (quién eres) y autorización (qué puedes hacer) en todo el sistema.
BCrypt	Algoritmo de cifrado obligatorio para proteger y guardar contraseñas de forma segura en la base de datos.
JSON Web Token (JWT)	Biblioteca para generar tokens de sesión seguros y sin estado, permitiendo que el usuario permanezca logueado sin usar sesiones de servidor.
Control de Acceso (RBAC)	Técnica para restringir el acceso a ciertas partes de la API basándose en los roles del usuario (ej: Administrador vs Cliente).
VII. Despliegue (DevOps)	
Maven / Gradle	Herramientas para gestionar todas las bibliotecas del proyecto y empaquetar el código en un archivo .jar ejecutable.
Docker / Docker Compose	Tecnología para "contenerizar" la aplicación y base de datos, asegurando que funcionen igual en cualquier PC, e iniciarlas con un solo comando.
Ngrok / Localtunnel	Herramientas que crean un túnel seguro para exponer el servidor local a internet, dándote una URL pública temporal para pruebas externas.
Railway / Render / Koyeb	Servicios en la nube gratuitos (o con capa gratuita) para alojar el proyecto en producción de forma permanente y con una URL fija.
VIII. Gestión de Código	
GitHub	Plataforma obligatoria para almacenar el código, controlar versiones, trabajar en equipo y evidenciar el progreso.
Branches (Ramas) / Commits	Prácticas de Git para organizar el desarrollo de nuevas funciones y registrar un historial claro de los cambios del proyecto.